

|                  |                                                              |
|------------------|--------------------------------------------------------------|
| EXPNO:5<br>DATE: | <b>Cloud Object Storage Manager Using Python &amp; MinIO</b> |
|------------------|--------------------------------------------------------------|

### Aim:

To develop a Python-based application that interacts with a local MinIO server (an S3-compatible cloud storage emulator) to understand object storage concepts such as buckets, unique object keys, and metadata tracking.

### Problem Statement:

Develop a Python application that mimics cloud object storage behavior using an open-source MinIO server running locally inside a VirtualBox VM, communicating via the official MinIO Python API. The application must implement:

- Create Bucket – initialize a flat container on the MinIO server.
- Upload Object – upload a local file into a bucket with a unique key.
- Download Object – retrieve an object using its unique key.
- List Bucket Contents – display objects with metadata (size, last modified).
- Delete Object – remove an object using its unique key.

### Pre-Requisites:

- Windows 10 Host Machine
- Oracle VM VirtualBox (Open Source)
- Ubuntu Desktop ISO image (Open Source)

### Architecture:



## Procedure:

### Step 1-2: Update Ubuntu & Install Python

```
sudo apt upgrade -y
sudo apt install python3 python3-pip -y
python3 --version
pip3 --version
```

### Step 3: Install MinIO Server

```
mkdir ~/minio && cd ~/minio
wget https://dl.min.io/server/minio/release/linux-amd64/minio
chmod +x minio
sudo mv minio /usr/local/bin/
minio --version
```

### Step 4-5: Create Storage Folder & Start Server

```
mkdir ~/minio-data
export MINIO_ROOT_USER=minioadmin
export MINIO_ROOT_PASSWORD=minioadmin
minio server ~/minio-data --console-address ":9001"
```

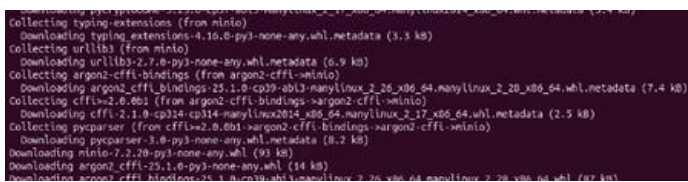
API: <http://127.0.0.1:9000> | Console: <http://127.0.0.1:9001> | Login: minioadmin / minioadmin

### Step 6: Open MinIO Console

Visit <http://localhost:9001> and log in. Initially there are no buckets.

### Step 7-8: Create Project & Virtual Environment

```
mkdir ObjectStorageSimulator && cd ObjectStorageSimulator
python3 -m venv venv
source venv/bin/activate
```



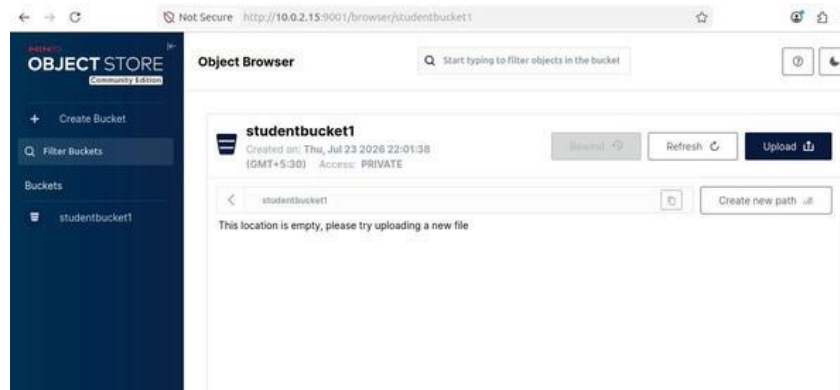
```
Collecting minio
  Downloading minio-7.2.20-py3-none-any.whl (93 kB)
Collecting typing-extensions<4.10.0,>=4.10.0
  Downloading typing_extensions-4.10.0-py3-none-any.whl.metadata (3.3 kB)
Collecting urllib3
  Downloading urllib3-2.2.0-py3-none-any.whl.metadata (6.9 kB)
Collecting argon2-cffi-bindings
  Downloading argon2_cffi_bindings-25.1.0-cp39-cp310-manylinux_2_28_x86_64_manylinux_2_28_x86_64.whl.metadata (7.4 kB)
Collecting cffi>=2.0.0
  Downloading cffi-2.0.0-cp314-cp314-manylinux2014_x86_64_manylinux_2_17_x86_64.whl.metadata (2.5 kB)
Collecting pycparser
  Downloading pycparser-3.0-py3-none-any.whl.metadata (8.7 kB)
Installing collected packages: typing-extensions, urllib3, argon2-cffi-bindings, cffi, pycparser, minio
Successfully installed minio-7.2.20 pycparser-3.0 typing-extensions-4.10.0 urllib3-2.2.0
```

### Step 9: Install MinIO Python SDK

```
pip install minio
```



Verify in the MinIO console — the new bucket appears with no objects yet:

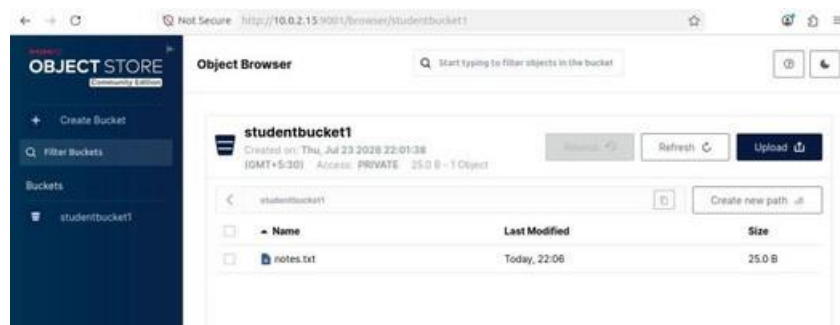


### Step 11: Upload an Object

```
client.fput_object("studentbucket", "notes.txt", "sample.txt")  
print("Upload Successful")
```

```
collecting pycryptodome-3.22.0-cp37-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (3.4 kB)  
collecting typing_extensions (from minio)  
  Downloading typing_extensions-4.16.0-py3-none-any.whl.metadata (3.3 kB)  
collecting urllib3 (from minio)  
  Downloading urllib3-2.7.0-py3-none-any.whl.metadata (6.9 kB)  
collecting argon2-cffi-bindings (from argon2-cffi->minio)  
  Downloading argon2_cffi_bindings-25.1.0-cp39-abi3-manylinux_2_28_x86_64.manylinux_2_28_x86_64.whl.metadata (7.4 kB)  
collecting cffi>=2.0.0b1 (from argon2-cffi-bindings->argon2-cffi->minio)  
  Downloading cffi-2.1.0-cp314-cp314-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (2.5 kB)  
collecting pycparser (from cffi>=2.0.0b1->argon2-cffi-bindings->argon2-cffi->minio)  
  Downloading pycparser-3.0-py3-none-any.whl.metadata (8.2 kB)  
collecting minio-7.2.20-py3-none-any.whl (93 kB)  
  Downloading minio-7.2.20-py3-none-any.whl (93 kB)  
collecting argon2-cffi-25.1.0-py3-none-any.whl (14 kB)
```

Verify in the console — notes.txt now appears inside the bucket:



## Step 12: Download an Object

Modify main.py to replace the upload code with:

Replace code.

```
<> Python
from minio import Minio

client = Minio(
    "localhost:9000",
    access_key="minioadmin",
    secret_key="minioadmin",
    secure=False
)

client.fget_object(
    "studentbucket1",
    "notes.txt",
    "downloaded.txt"
)

print("Download Successful")
```

## Result:

Thus, a Python application was developed to perform bucket creation, object upload, and object download using the MinIO S3-compatible local object storage server.

